



University
of Glasgow

W3-04: Apache Spark Basics

COMPSCI4064 (H) and COMPSCI5088 (M)

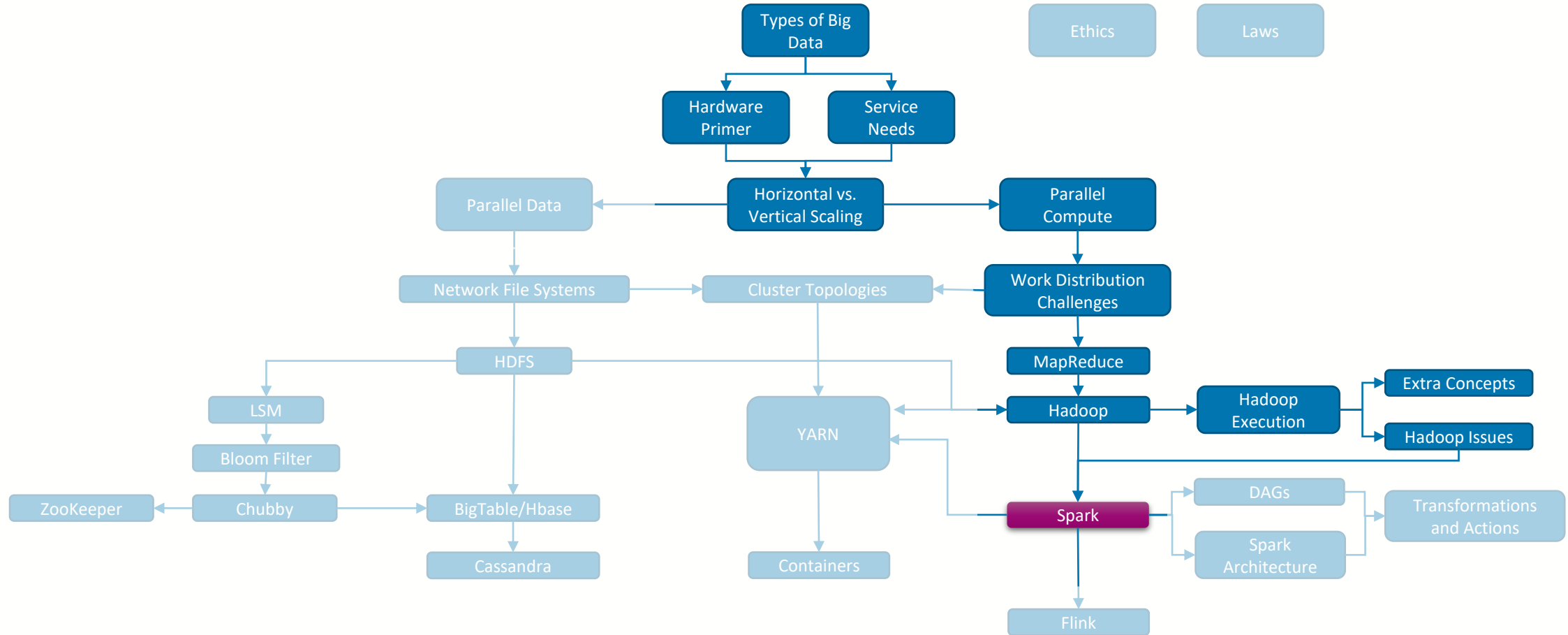
Dr. Richard McCreddie

**WORLD
CHANGING
GLASGOW**

**A WORLD
TOP 100
UNIVERSITY**



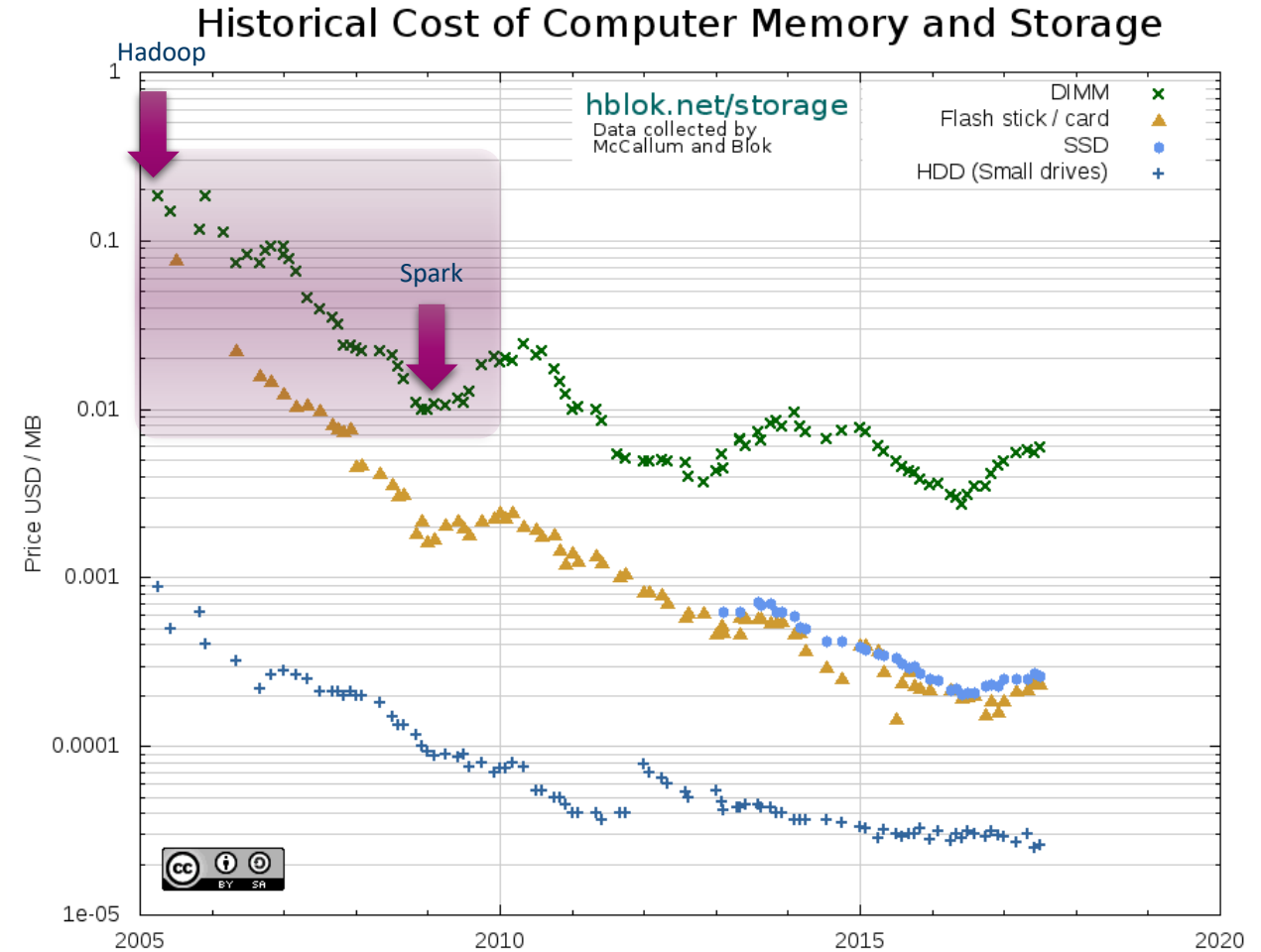
Where are we?





Hardware Opportunities

- In comparison to when Hadoop was designed, in 2009 **main memory costs had reduced 20x**
 - This meant that **commodity machines** could be expected to have a lot of it
- Developers around the world were working on better tools that could make use of this much larger resource
 - Those interested in **horizontal scaling** were no exception



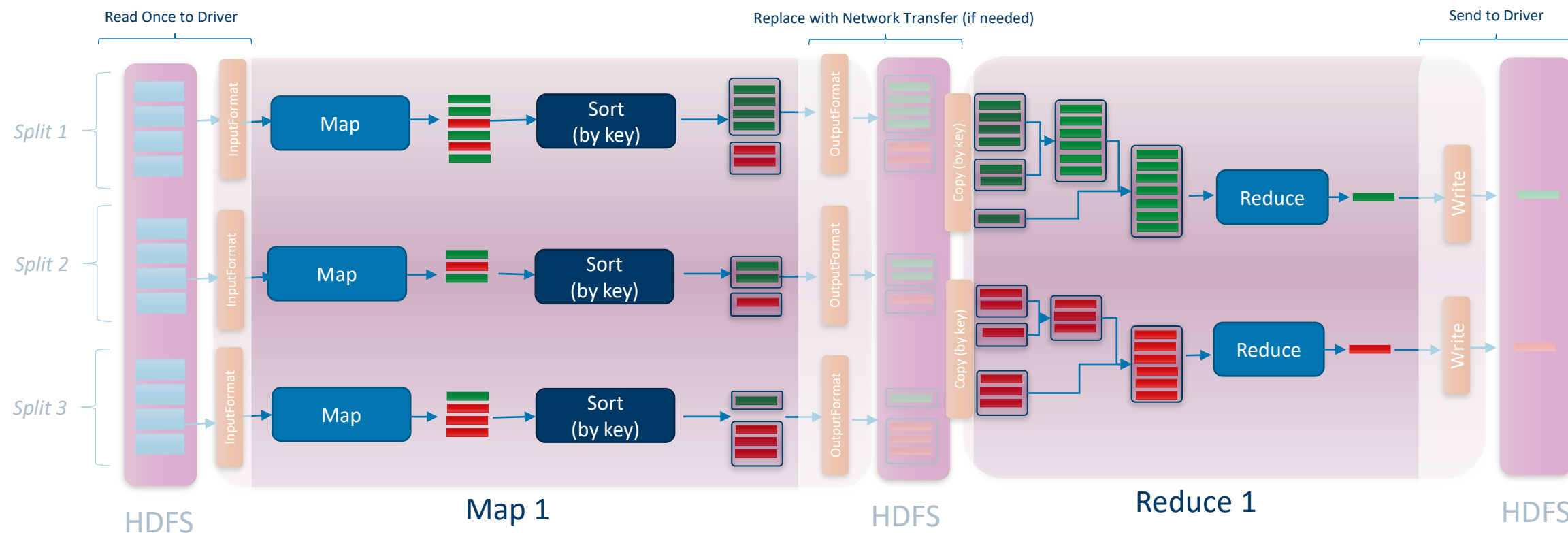


The Birth of Apache Spark

- **Apache Spark** was initially a class project at the University of California, Berkeley, founded in 2009, just over three years after the open source release of Hadoop
 - The first version was open-sourced a year later in 2010
- Apache Spark was designed as a solution to the **short-comings of MapReduce and Hadoop**
 - In short, Apache Spark started life as **MapReduce+ in Memory**

Eliminating Writes

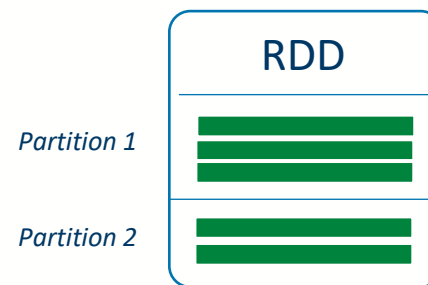
- The core motivation for Spark at the start was to **eliminate read/write operations** to persistent storage
 - Results in significantly **less overheads**
 - **Chaining** maps and reduces becomes **cheap and easy**



Resilient Distributed Datasets (RDDs)

- What makes this possible is how Spark stores its data in memory, using a structure called a **Resilient Distributed Dataset (RDD)**
 - An RDD is a **collection of records**, think of it like a Table
 - It **may or may not have a schema** (table header)
 - An RDD can be **partitioned**, and those partitions can be sent over the network to other machines
 - When an RDD is loaded it is **automatically partitioned**
 - RDD partitions can be **processed in parallel**

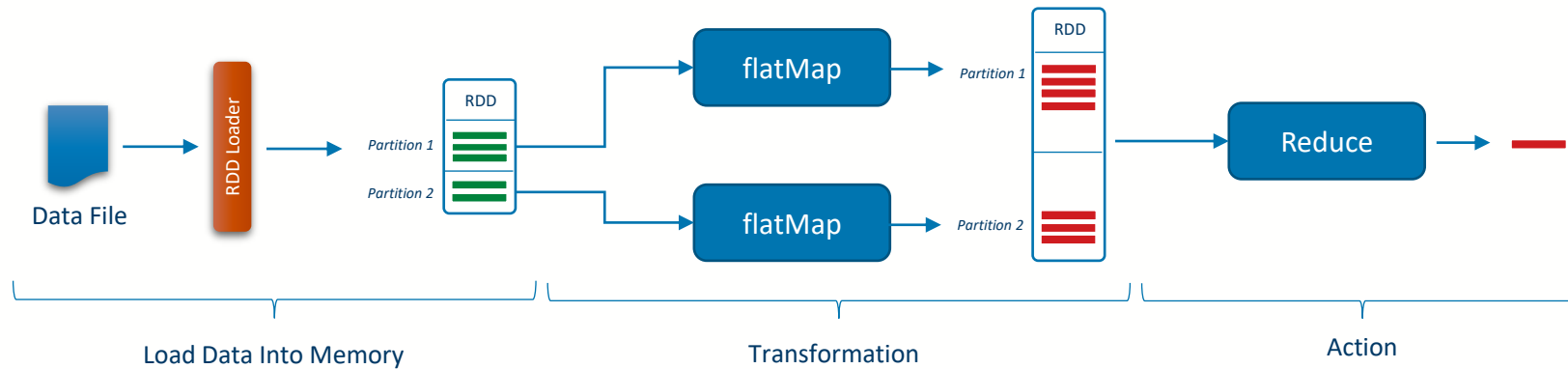
Firstname	Lastname	Degree
Joe	Blogs	CS
John	Doe	Business
Kate	Brown	Business
David	Smith	Math
Sara	Williams	Physics





Basic Spark Execution Flow

- Lets consider what the equivalent to a **MapReduce job** would look like...



- Data is now read from file(s) directly (no HDFS) into memory
- There is no concept of Input Spits, just data partitions
- No writes between flatMap (Map) and Reduce
- No end-step write, data is sent to the memory of the user's machine (usually)



Transformations and Actions

- Tasks in Spark are categorized into **two main types**
 - **Transformations**: These convert one RDD into another RDD
 - **Actions**: These either return an RDD (or metadata about an RDD) to the user's machine or writes the RDD to persistent storage
- Transformations can be **chained together in any way that the developer wants**
 - No more enforced Map->Reduce structures





Base Functionality (2010)

FlatMap (Transformation)

- The equivalent to a **Map** in MapReduce
 - Note however that it is **key-less**
- ***flatMap**(record1) -> List<record2>*

Reduce (Action)

- Similar to Reduce in MapReduce, but acts in a **pair-wise manner** rather than on lists
- ***reduce**(record1,record1) -> record1*
- When called on an RDD, reduce will be **applied recursively** until a single record is left
 - As an action, this final record will be returned to the user



Base Functionality (2010)

Collect (Action)

- The collect action simply returns an RDD to the user

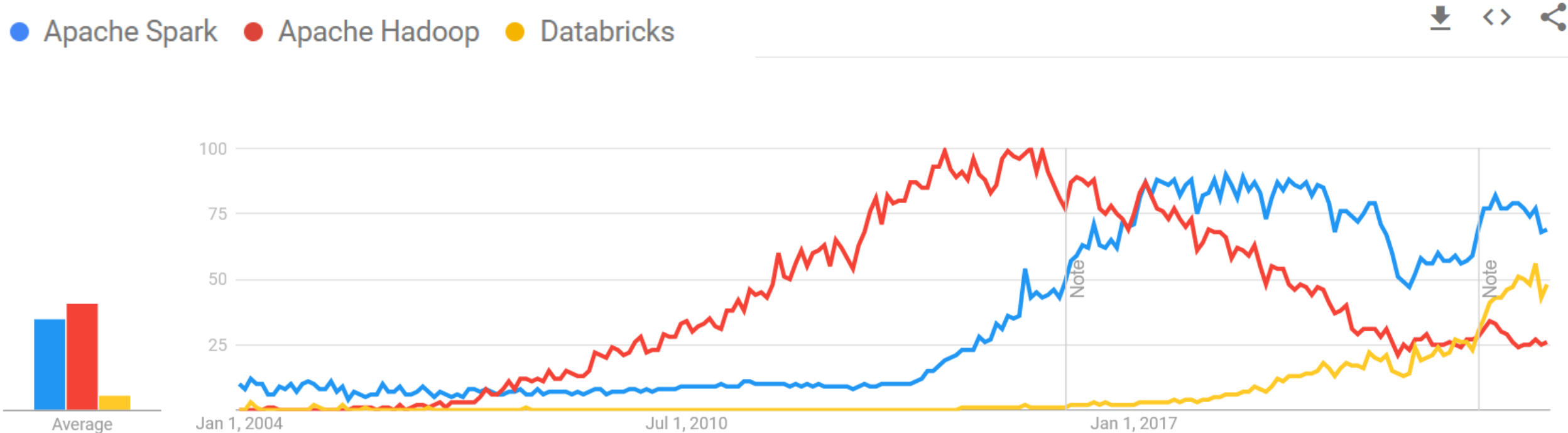
- `collect()`

- This allows Spark to have jobs with no reduces

... this was only the beginning

Spark Popularity

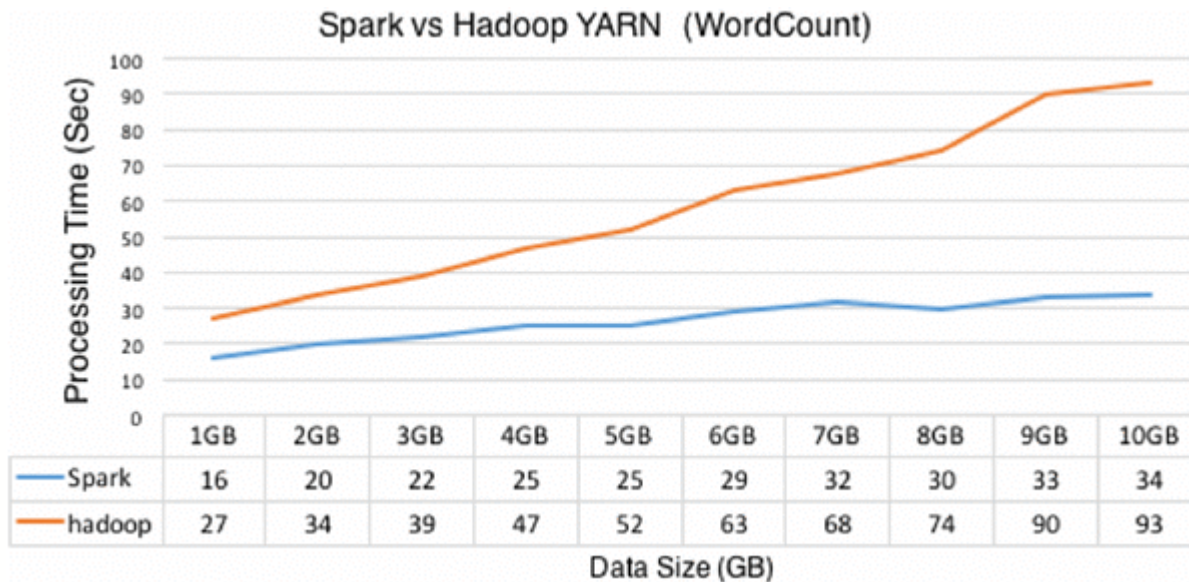
- Since around 2014, Apache Spark exploded in popularity, and overtook Hadoop in 2017
- Looks like Spark popularity has flattened in recent years, but this is due to people using Spark via an intermediate product, e.g. Databricks





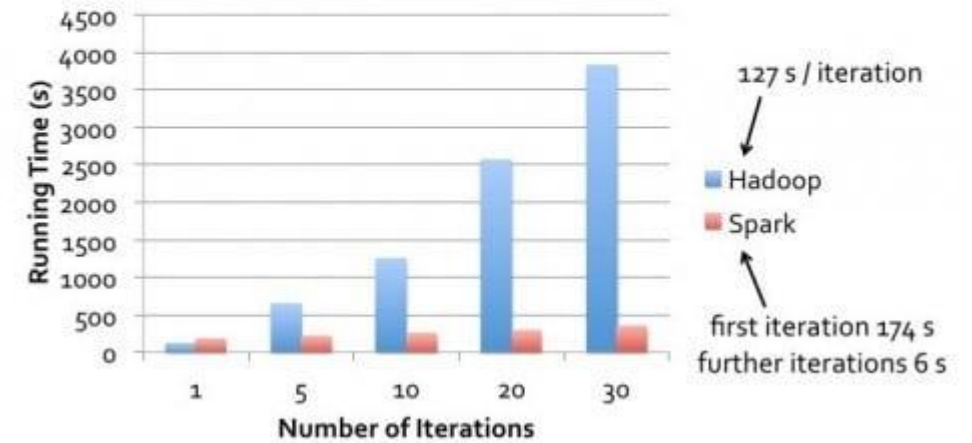
Why the Hype?

- In short, Spark is faster for most workloads



(Lower is better)

Logistic Regression Performance



(Lower is better)



University
of Glasgow

Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers



@UofGlasgow